# Матричное дифференцирование

шпаргалка к занятию 10: обратное распространение ошибки

Машинное обучение, 3 курс ФАЛТ МФТИ, осень 2026

Градиентный спуск $W \leftarrow W - \eta \nabla_W L$ нуждается в градиенте функции потерь по всем весам. Весов сотни тысяч, и считать производные по одной нельзя: нужны правила, которые дают сразу весь градиент в виде вектора или матрицы. Эта шпаргалка собирает такие правила, выводит каждое и доводит до формул обратного прохода слоев нейронной сети.

## 1. Четыре способа считать градиент

1. **Формула в явном виде.** Выписать $\nabla f$ руками по правилам ниже. Хорошо для простых функций; для сети из десятков слоев формула неподъемна.
2. **Символьное дифференцирование** (SymPy, Mathematica). Нужна явная формула функции; результат разрастается, одни и те же куски считаются повторно.
3. **Конечные разности.** $\frac{\partial f}{\partial x_i} \approx \frac{f(x+\varepsilon e_i) - f(x-\varepsilon e_i)}{2\varepsilon}$: два вычисления функции на каждую координату, для $10^5$ весов — $2 \cdot 10^5$ прямых проходов на один шаг. Ошибка порядка $\varepsilon^2$. Негодно для обучения, но незаменимо для *проверки*: сравнить с аналитическим градиентом, относительная ошибка меньше $10^{-6}$ — градиент верен.
4. **Автоматическое дифференцирование.** Функцию записывают графом простых операций, у каждой известна локальная производная; обратный проход по цепному правилу дает градиент по всем аргументам за время, сравнимое с одним вычислением функции. Это и есть обратное распространение ошибки; так работает `autograd` в PyTorch.

## 2. Определения: что считается производной

Пусть $f\colon D \to E$. Что такое $\frac{\partial f}{\partial x}$, зависит от того, что на входе и на выходе.

| вход $D$ | выход $E$ | производная | название |
|---|---|---|---|
| $\mathbb{R}$ | $\mathbb{R}$ | число $f'(x)$ | производная |
| $\mathbb{R}^n$ | $\mathbb{R}$ | вектор $\nabla f(x) \in \mathbb{R}^n$, $(\nabla f)_i = \frac{\partial f}{\partial x_i}$ | градиент |
| $\mathbb{R}^n$ | $\mathbb{R}^m$ | матрица $J \in \mathbb{R}^{m \times n}$, $J_{ij} = \frac{\partial f_i}{\partial x_j}$ | матрица Якоби |
| $\mathbb{R}^{m \times n}$ | $\mathbb{R}$ | матрица $\frac{\partial f}{\partial A} \in \mathbb{R}^{m \times n}$, $\left(\frac{\partial f}{\partial A}\right)_{ij} = \frac{\partial f}{\partial a_{ij}}$ | матричный градиент |
| $\mathbb{R}^n$ | $\mathbb{R}$, вторые производные | матрица $H \in \mathbb{R}^{n \times n}$, $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ | матрица Гессе |

**Правило формы.** Градиент по аргументу имеет ту же форму, что аргумент: по вектору — вектор, по матрице $W \in \mathbb{R}^{n \times m}$ — матрица $n \times m$. Это главный способ проверить формулу: размеры обязаны сойтись.

**Градиент через линеаризацию.** Для малого приращения $\Delta$

$$f(x + \Delta) = f(x) + \langle \nabla f(x), \Delta \rangle + o(\|\Delta\|),$$

где $\langle a, b \rangle = \sum_i a_i b_i = a^\top b$ для векторов и $\langle A, B \rangle = \operatorname{tr}(A^\top B) = \sum_{ij} a_{ij} b_{ij}$ для матриц. Градиент — тот вектор (матрица), который стоит в скалярном произведении с приращением. Этим определением удобно пользоваться: раскрыть $f(x+\Delta)$, собрать линейные по $\Delta$ слагаемые в вид $\langle$что-то$, \Delta\rangle$, и «что-то» — градиент.

## 3. Цепное правило

Скалярный случай: $(f(g(x)))' = f'(g(x))\, g'(x)$. В многомерном случае производные — матрицы Якоби, и они *перемножаются*, причем порядок важен.

**Цепное правило для матриц Якоби**

Пусть $g\colon \mathbb{R}^n \to \mathbb{R}^p$, $f\colon \mathbb{R}^p \to \mathbb{R}^m$, $h = f \circ g$. Тогда

$$J_h(x) = J_f(g(x))\, J_g(x), \qquad (m \times n) = (m \times p)(p \times n).$$

Для скалярной $f$ ($m = 1$) матрица Якоби $J_f$ — строка $(\nabla f)^\top$, и градиент композиции

$$\nabla h(x) = J_g(x)^\top \nabla f(g(x)), \qquad (n \times 1) = (n \times p)(p \times 1).$$

**Вывод.** По определению $h_i(x) = f_i(g_1(x), \ldots, g_p(x))$. Производная по $x_j$ по правилу дифференцирования сложной функции нескольких переменных:

$$\frac{\partial h_i}{\partial x_j} = \sum_{k=1}^{p} \frac{\partial f_i}{\partial g_k} \frac{\partial g_k}{\partial x_j} = \sum_{k=1}^{p} (J_f)_{ik} (J_g)_{kj} = (J_f J_g)_{ij}.$$

Для градиента скалярной функции транспонируем строку $J_h = \nabla f^\top J_g$ и получаем $\nabla h = J_g^\top \nabla f$.

**Обратный проход — это цепное правило справа налево.** Сеть $L = \ell(f_K(\ldots f_2(f_1(x))))$, обозначим выход слоя $k$ через $h_k$. Градиент по входу слоя $k$:

$$\nabla_{h_{k-1}} L = J_{f_k}^\top \nabla_{h_k} L.$$

Каждый слой получает градиент по своему выходу и умножает на транспонированную матрицу Якоби, отдавая градиент по входу. Саму матрицу Якоби никогда не строят: для каждого слоя произведение $J^\top g$ записывается напрямую (раздел 6).

## 4. Правила для векторов

Ниже $x \in \mathbb{R}^n$, $a, c \in \mathbb{R}^n$ — постоянные векторы, $A \in \mathbb{R}^{n \times n}$ — постоянная матрица.

**Таблица градиентов**

| | | | |
|---|---|---|---|
| $f(x) = a^\top x = \langle a, x \rangle$ | $\nabla f = a$ | $f(x) = x^\top x = \|x\|^2$ | $\nabla f = 2x$ |
| $f(x) = x^\top A x$ | $\nabla f = (A + A^\top)\, x$ | $f(x) = x^\top A x,\ A = A^\top$ | $\nabla f = 2Ax$ |
| $f(x) = \|x\|$ | $\nabla f = x / \|x\|$ | $f(x) = \frac{1}{2}\|Ax - b\|^2$ | $\nabla f = A^\top (Ax - b)$ |
| $f(x) = u(x)^\top v(x)$ | $\nabla f = J_u^\top v + J_v^\top u$ | $f(x) = \varphi(a^\top x)$ | $\nabla f = \varphi'(a^\top x)\, a$ |

**Вывод 1:** $a^\top x$**.** $f = \sum_i a_i x_i$, $\frac{\partial f}{\partial x_j} = a_j$, значит $\nabla f = a$. Через линеаризацию: $f(x+\Delta) = a^\top x + a^\top \Delta = f(x) + \langle a, \Delta \rangle$.

**Вывод 2:** $x^\top A x$**.** Раскроем: $f = \sum_{i,j} a_{ij} x_i x_j$. Координата $x_k$ входит в слагаемые с $i = k$ и с $j = k$:

$$\frac{\partial f}{\partial x_k} = \sum_j a_{kj} x_j + \sum_i a_{ik} x_i = (Ax)_k + (A^\top x)_k, \qquad \nabla f = (A + A^\top) x.$$

При симметричной $A$ это $2Ax$; при $A = I$ получаем $\nabla \|x\|^2 = 2x$.

**Вывод 3:** $\|x\|$**.** Это композиция $\sqrt{\cdot}$ и $x^\top x$: $\nabla \sqrt{g} = \frac{1}{2\sqrt{g}} \nabla g = \frac{2x}{2\|x\|} = \frac{x}{\|x\|}$.

**Вывод 4:** $\frac{1}{2}\|Ax - b\|^2$**.** Обозначим $r = Ax - b$, тогда $f = \frac{1}{2} r^\top r$, $\nabla_r f = r$, матрица Якоби $r$ по $x$ равна $A$; по цепному правилу $\nabla_x f = A^\top r = A^\top (Ax - b)$. Это градиент MSE линейной регрессии; приравняв его к нулю, получаем нормальное уравнение $A^\top A x = A^\top b$.

**Вывод 5:** $\varphi(a^\top x)$**.** Внешняя функция скалярная, $\varphi'(a^\top x)$ — число; внутренняя $a^\top x$ дает градиент $a$. Так считают градиент логистической регрессии: $\nabla_w \sigma(w^\top x) = \sigma(1 - \sigma)\, x$.

## 5. Правила для матриц

Пусть $f$ — скалярная функция матрицы $X$, $A$, $B$ — постоянные матрицы подходящих размеров.

**Таблица матричных градиентов**

| | |
|---|---|
| $f(X) = \mathrm{tr}(A^\top X) = \langle A, X\rangle = \sum_{ij} a_{ij}x_{ij}$ | $\nabla_X f = A$ |
| $f(X) = u^\top X v$ | $\nabla_X f = uv^\top$ |
| $f(X) = \mathrm{tr}(X)$ | $\nabla_X f = I$ |
| $f(X) = \|X\|_F^2 = \mathrm{tr}(X^\top X) = \sum_{ij} x_{ij}^2$ | $\nabla_X f = 2X$ |
| $f(X) = \frac{1}{2}\|XW - Y\|_F^2$ | $\nabla_X f = (XW - Y)\,W^\top, \quad \nabla_W f = X^\top(XW - Y)$ |
| $f(X) = \ln\det X,\ X \succ 0$ | $\nabla_X f = X^{-\top}$ |

**Вывод 1:** $u^\top X v$**.** $f = \sum_{ij} u_i x_{ij} v_j$, $\frac{\partial f}{\partial x_{ij}} = u_i v_j = (uv^\top)_{ij}$.

**Вывод 2:** $\frac{1}{2}\|XW - Y\|_F^2$ **по** $W$**.** Через линеаризацию. Обозначим $R = XW - Y$. При $W \to W + \Delta$ остаток $R \to R + X\Delta$:

$$\tfrac{1}{2}\|R + X\Delta\|_F^2 = \tfrac{1}{2}\|R\|_F^2 + \langle R, X\Delta\rangle + o(\|\Delta\|).$$

Переносим $X$ на другую сторону скалярного произведения:

$$\langle R, X\Delta\rangle = \mathrm{tr}(R^\top X\Delta) = \mathrm{tr}\left((X^\top R)^\top \Delta\right) = \langle X^\top R, \Delta\rangle, \qquad \nabla_W f = X^\top R = X^\top(XW - Y).$$

По $X$ аналогично: $\langle R, \Delta W\rangle = \mathrm{tr}(R^\top \Delta W) = \mathrm{tr}(WR^\top \Delta) = \langle RW^\top, \Delta\rangle$, откуда $\nabla_X f = RW^\top$.

**Полезные тождества со следом** (они и дают «перенос матрицы на другую сторону»):

$$\mathrm{tr}(AB) = \mathrm{tr}(BA), \qquad \mathrm{tr}(ABC) = \mathrm{tr}(BCA) = \mathrm{tr}(CAB), \qquad \mathrm{tr}(A^\top) = \mathrm{tr}(A),$$
$$\langle A, BC\rangle = \langle B^\top A, C\rangle = \langle AC^\top, B\rangle.$$

## 6. Обратный проход слоев: выводы

Теперь главное. Слой получает $G = \frac{\partial L}{\partial Y}$ — градиент потери по своему выходу $Y$ (та же форма, что $Y$) и должен вернуть градиенты по параметрам и по входу. Везде пачка из $n$ объектов записана по строкам: $X \in \mathbb{R}^{n \times n_{in}}$.

### 6.1. Полносвязный слой $Y = XW + b$

$Y \in \mathbb{R}^{n \times n_{out}}$, $W \in \mathbb{R}^{n_{in} \times n_{out}}$, $b \in \mathbb{R}^{n_{out}}$, $Y_{ij} = \sum_k X_{ik}W_{kj} + b_j$.

**Результат**

$$\frac{\partial L}{\partial W} = X^\top G \ \ (n_{in} \times n_{out}), \qquad \frac{\partial L}{\partial b} = \sum_{i=1}^{n} G_{i\cdot} = \mathbf{1}^\top G \ \ (n_{out}), \qquad \frac{\partial L}{\partial X} = G\,W^\top \ \ (n \times n_{in}).$$

**Вывод по элементам.** Потеря зависит от $W_{kj}$ через все $Y_{ij}$ столбца $j$, и $\frac{\partial Y_{ij}}{\partial W_{kj}} = X_{ik}$:

$$\frac{\partial L}{\partial W_{kj}} = \sum_i \frac{\partial L}{\partial Y_{ij}}\frac{\partial Y_{ij}}{\partial W_{kj}} = \sum_i G_{ij}X_{ik} = \sum_i (X^\top)_{ki}G_{ij} = (X^\top G)_{kj}.$$

Сдвиг $b_j$ входит в каждую строку $i$ с коэффициентом 1: $\frac{\partial L}{\partial b_j} = \sum_i G_{ij}$. Вход $X_{ik}$ входит во все $Y_{ij}$ строки $i$ с коэффициентом $W_{kj}$:

$$\frac{\partial L}{\partial X_{ik}} = \sum_j G_{ij}W_{kj} = \sum_j G_{ij}(W^\top)_{jk} = (GW^\top)_{ik}.$$

**Вывод через линеаризацию** (короче). $W \to W + \Delta$ дает $Y \to Y + X\Delta$, и $L \to L + \langle G, X\Delta\rangle = L + \langle X^\top G, \Delta\rangle$, откуда $\frac{\partial L}{\partial W} = X^\top G$. Так же $X \to X + \Delta$: $\langle G, \Delta W\rangle = \langle GW^\top, \Delta\rangle$.

**Проверка размеров.** $X^\top G$: $(n_{in} \times n)(n \times n_{out}) = n_{in} \times n_{out}$ — как $W$. $GW^\top$: $(n \times n_{out})(n_{out} \times n_{in}) = n \times n_{in}$ — как $X$. Если размеры не сходятся, формула неверна; если сходятся, но транспонирование перепутано, их поймает численная проверка.

### 6.2. Поэлементные слои $Y_{ij} = \varphi(X_{ij})$

Каждый выход зависит только от своего входа, матрица Якоби диагональная, и

$$\frac{\partial L}{\partial X_{ij}} = G_{ij}\,\varphi'(X_{ij}), \qquad \frac{\partial L}{\partial X} = G \odot \varphi'(X)$$

($\odot$ — поэлементное произведение). Производные:

| | | |
|---|---|---|
| ReLU | $\varphi = \max(z, 0)$ | $\varphi' = [z > 0]$ |
| Leaky ReLU | $\varphi = \max(z, \alpha z)$ | $\varphi' = [z > 0] + \alpha[z \le 0]$ |
| сигмоида | $\varphi = \sigma(z) = \dfrac{1}{1+e^{-z}}$ | $\varphi' = \sigma(z)\,(1-\sigma(z))$ |
| tanh | $\varphi = \tanh z$ | $\varphi' = 1 - \tanh^2 z$ |

**Вывод для сигмоиды.** По правилу для частного

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\,\big(1-\sigma(z)\big), \quad \text{так как} \quad \frac{e^{-z}}{1+e^{-z}} = 1 - \frac{1}{1+e^{-z}}.$$

В коде выход $\sigma(z)$ уже посчитан на прямом проходе, поэтому производную берут из него. Наибольшее значение $\sigma' = \frac{1}{4}$ при $z = 0$: за каждый слой с сигмоидой градиент уменьшается не меньше чем вчетверо — затухание градиента.

### 6.3. Софтмакс и кросс-энтропия

Оценки $z \in \mathbb{R}^K$ одного объекта, $p_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$, верный класс $y$, потеря $\ell = -\ln p_y = -z_y + \ln \sum_j e^{z_j}$.

**Результат**

$$\frac{\partial \ell}{\partial z_k} = p_k - [k = y], \qquad \text{для пачки из } n \text{ объектов и средней потери} \quad \frac{\partial L}{\partial Z} = \frac{1}{n}\,(P - Y_{\text{one-hot}}).$$

**Вывод.** Первое слагаемое $-z_y$ дает $-[k = y]$. Второе: $\frac{\partial}{\partial z_k} \ln \sum_j e^{z_j} = \dfrac{e^{z_k}}{\sum_j e^{z_j}} = p_k$. Сумма — $p_k - [k = y]$: сдвиг оценки класса $k$ тем больше, чем сильнее модель переоценила этот класс. Софтмакс и кросс-энтропию считают одним узлом именно потому, что вместе их градиент проще, чем у каждого по отдельности: матрица Якоби софтмакса $\frac{\partial p_k}{\partial z_j} = p_k([k = j] - p_j)$ при свертке с $-1/p_y$ сворачивается в $p - e_y$.

**Устойчивость.** Софтмакс не меняется от сдвига всех $z_k$ на одно число $c$: $\frac{e^{z_k - c}}{\sum_j e^{z_j - c}} = \frac{e^{z_k}}{\sum_j e^{z_j}}$. Берут $c = \max_k z_k$, тогда экспоненты не переполняются.

### 6.4. Два слоя подряд: цепное правило в действии

Сеть $Z = \text{ReLU}(XW_1 + b_1)\,W_2 + b_2$, потеря $L$. Обозначим $H_{\text{pre}} = XW_1 + b_1$, $H = \text{ReLU}(H_{\text{pre}})$. Обратный проход, справа налево:

$$\begin{aligned}
G_Z &= \frac{\partial L}{\partial Z} = \tfrac{1}{n}(P - Y_{\text{one-hot}}) && \text{потеря;}\\
\frac{\partial L}{\partial W_2} &= H^\top G_Z, \quad \frac{\partial L}{\partial b_2} = \mathbf{1}^\top G_Z, \quad G_H = G_Z W_2^\top && \text{слой 2;}\\
G_{\text{pre}} &= G_H \odot [H_{\text{pre}} > 0] && \text{ReLU;}\\
\frac{\partial L}{\partial W_1} &= X^\top G_{\text{pre}}, \quad \frac{\partial L}{\partial b_1} = \mathbf{1}^\top G_{\text{pre}}, \quad G_X = G_{\text{pre}} W_1^\top && \text{слой 1.}
\end{aligned}$$

$G_X$ — градиент по картинке: он не нужен для обучения, но дает атаки на сеть и карты важности пикселей.

## 7. Как проверять себя

1. **Размеры.** Градиент по параметру — той же формы, что параметр. Перемножаемые матрицы должны стыковаться по внутреннему размеру.
2. **Скалярный случай.** Положите $n = n_{in} = n_{out} = 1$: формула должна превратиться в школьную, $\mathrm{d}(xw)/\mathrm{d}w = x$.
3. **Численно.** Центральная разность с $\varepsilon = 10^{-5}$ и относительная ошибка $\max \dfrac{|a-b|}{|a|+|b|}$; меньше $10^{-6}$ — верно, около $1$ — перепутано транспонирование или знак.
4. **Линеаризация.** Если формула не выводится по элементам, раскройте $f(x+\Delta)$ и соберите $\langle \cdot, \Delta \rangle$: то, что стоит слева, и есть градиент.

## 8. Что почитать

- Записки cs231n: Vector, Matrix, and Tensor Derivatives (E. Yang) и Backpropagation for a Linear Layer (J. Johnson).
- K. Petersen, M. Pedersen, The Matrix Cookbook — справочник формул.
- I. Goodfellow и др., Deep Learning, раздел 6.5 — автоматическое дифференцирование.
- matrixcalculus.org — считает матричные производные символьно; удобно для самопроверки.